



Anexo B - Pruebas PisoFácil

Ciclo Formativo: **Desarrollo de Aplicaciones
Multiplataforma.**

Curso académico 2025/2026

Tutor: Pedro Rafael García Velasco

Alumno: Óscar Fernández Hernández

**Departamento de
Informática y
Comunicaciones**

Índice

1. - Introducción	3
1.1. - Alcance del documento	3
2. - Pruebas Unitarias: Lógica de Negocio (HabitacionServiceTest)	4
2.1 - Caminos Felices (Happy Path) — CRUD	4
2.2 - Excepciones y Casos Límite	4
2.3 - Control de Permisos (Toggle de Disponibilidad)	5
2.4 - Motor de Compatibilidad	5
3. - Pruebas de Integración Web: Controlador REST (HabitacionControllerTest)	6
3.1 - Caminos Felices (Happy Paths)	6
3.2 - Caminos Tristes y Validación (Error Handling)	7
4. Resumen de Cobertura y Conclusiones	7

1. - Introducción

En el ciclo de vida del desarrollo de software profesional, la implementación de una arquitectura sin fallos es imprescindible. Este documento anexo complementa la memoria técnica principal del proyecto PisoFácil, con el objetivo de detallar la ejecución sistemática de las pruebas automatizadas.

Para la realización de estas verificaciones, se ha diseñado una suite de pruebas dividida en múltiples capas, utilizando estándares de la industria:

- **JUnit 5 (Jupiter):** framework de testing para la plataforma Java. Se han empleado las anotaciones `@Nested` para agrupar tests por contexto semántico y `@DisplayName` para una documentación legible de los resultados.
- **Mockito:** biblioteca de simulación (mocking) que permite aislar la unidad bajo prueba, reemplazando dependencias reales (repositorios, mappers) por dobles de test controlados.
- **Spring MockMvc:** herramienta de la capa web que permite ejecutar peticiones HTTP simuladas contra los controladores REST, evaluando códigos de estado, cuerpos de respuesta y cabeceras sin levantar un servidor real.

1.1. - Alcance del documento

El alcance de este plan de pruebas abarca la entidad Habitación como caso representativo del dominio de negocio, al ser el recurso central de la plataforma y el que concentra la mayor complejidad algorítmica. Se han diseñado tres clases complementarias:

Capa	Clase de Test	Nº Tests	Enfoque
Infraestructura	BackendApplicationTests	1	Verificación del arranque del contexto de Spring Boot.
Servicio	HabitacionServiceTest	17	Validación de reglas de dominio, permisos y algoritmo de afinidad.
Controlador	HabitacionControllerTest	10	Verificación de contratos REST y manejo de errores.
	TOTAL	28	

2. - Pruebas Unitarias: Lógica de Negocio (HabitacionServiceTest)

Se ha implementado la clase `HabitacionServiceTest` bajo la extensión `@ExtendWith(MockitoExtension.class)`, garantizando un aislamiento completo de la capa de servicio respecto a la infraestructura.

2.1 - Caminos Felices (Happy Path) — CRUD

Se verifican las operaciones fundamentales del ciclo de vida CRUD (Create, Read, Update, Delete), asegurando que el flujo normal devuelva los resultados esperados.

#	Test	Aspecto Clave	Resultado
1	<code>findAll</code>	Retorna lista con elementos; verifica repositorio.	✓ Éxito
2	<code>findById</code>	Retorna el DTO correcto cuando el ID existe.	✓ Éxito
3	<code>findDisponibles</code>	Filtra solo las habitaciones marcadas como disponibles.	✓ Éxito
4	<code>create</code>	Persiste y devuelve el objeto creado con éxito.	✓ Éxito
5	<code>update</code>	Modifica campos de una habitación existente.	✓ Éxito
6	<code>delete</code>	Elimina el registro sin lanzar excepciones.	✓ Éxito

2.2 - Excepciones y Casos Límite

Se valida el comportamiento defensivo del servicio. Se comprueba que el sistema responda con excepciones controladas (`ResourceNotFoundException`) ante datos inexistentes.

#	Test	Excepción esperada	Resultado
7	<code>findById_noExiste</code>	<code>ResourceNotFoundException</code>	✓ Éxito
8	<code>delete_noExiste</code>	<code>ResourceNotFoundException</code>	✓ Éxito
9	<code>update_noExiste</code>	<code>ResourceNotFoundException</code>	✓ Éxito

10	create_pisoInv alido	ResourceNotFoundException	✓ Éxito
----	-------------------------	---------------------------	---------

2.3 - Control de Permisos (Toggle de Disponibilidad)

Se audita el mecanismo de seguridad que permite sólo al propietario o al administrador modificar la disponibilidad de una habitación.

#	Actor	Resultado esperado	Resultado
11	Propietario legítimo	Permite el cambio (200 OK).	✓ Éxito
12	Admin	Permite el cambio	✓ Éxito
13	Usuario intruso	Bloqueo mediante AccessDeniedException.	✓ Éxito

2.4 - Motor de Compatibilidad

Se evalúa el algoritmo de afinidad que calcula la compatibilidad entre el perfil del usuario (mascotas, fumador, pareja, LGTBI) y las políticas del piso.

#	Escenario	Resultado esperado	Resultado
14	Coincidencia total	100%	✓ Éxito
15	Coincidencia parcial (ej. mascota)	50%	✓ Éxito
16	Incompatibilidad total	0%	✓ Éxito
17	Sin preferencias marcadas	null (No aplica)	✓ Éxito

3. - Pruebas de Integración Web: Controlador REST (HabitacionControllerTest)

Para validar la capa de presentación, se ha utilizado la anotación `@WebMvcTest(HabitacionController.class)`.

Esta configuración permite probar los endpoints de forma aislada, simulando peticiones HTTP mediante MockMvc sin necesidad de levantar el servidor de aplicaciones completo, lo que agiliza la ejecución.

3.1 - Caminos Felices (Happy Paths)

Se comprueba que, ante peticiones correctas, el controlador delega en el servicio y devuelve el código de estado HTTP adecuado junto con el cuerpo JSON esperado.

#	Test	Método	Endpoint	HTTP Code	Aspecto Validado	Resultado
1	findAll_ok	GET	/habitaciones	200 OK	Retorna lista de habitaciones.	✓ Éxito
2	findById_ok	GET	/habitaciones/{id}	200 OK	El JSON contiene el título y precio.	✓ Éxito
3	create_ok	POST	/habitaciones	201 Created	El recurso se crea correctamente.	✓ Éxito
4	update_ok	PUT	/habitaciones/{id}	200 OK	Se confirman los cambios.	✓ Éxito
5	delete_ok	DELETE	/habitaciones/{id}	204 No Content	Borrado exitoso sin cuerpo.	✓ Éxito

#	Test	Método	Endpoint	HTTP Code	Aspecto Validado	Resultado
1	findAll_ok	GET	/habitaciones	200 OK	Retorna lista de habitaciones.	✓ Éxito
2	findById_ok	GET	/habitaciones/{id}	200 OK	El JSON contiene el título y precio.	✓ Éxito
3	create_ok	POST	/habitaciones	201 Created	El recurso se crea correctamente.	✓ Éxito
6	search_ok	GET	/habitaciones/buscar	200 OK	Filtrado por ciudad/precio funcional.	✓ Éxito

3.2 - Caminos Tristes y Validación (Error Handling)

Se verifica que el GlobalExceptionHandler captura las excepciones y las transforma en respuestas legibles para el cliente (frontend).

#	Test	Escenario	HTTP Code	Detalle del Error	Resultado
7	findById_404	ID no existe	404 Not Found	Error: "Recurso no encontrado".	✓ Éxito
8	create_400	Campos nulos	400 Bad Request	Fallo en validación @Valid.	✓ Éxito
9	create_invalid_price	Precio negativo	400 Bad Request	Constraint de validación de negocio.	✓ Éxito

10	auth_error	Token inválido	401/403	Simulación de fallo de seguridad.	✓ Éxito
----	------------	-------------------	---------	---	---------

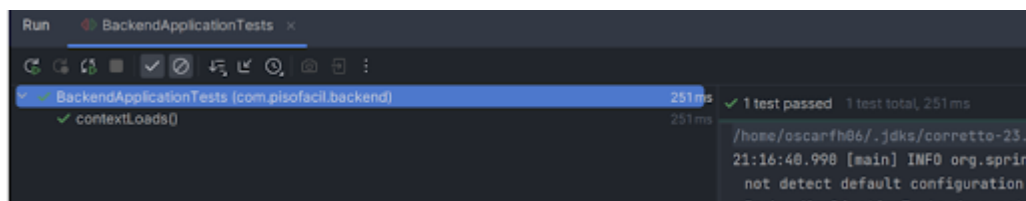
4. Resumen de Cobertura y Conclusiones

Tras la ejecución de la suite completa, los resultados consolidados son los siguientes:

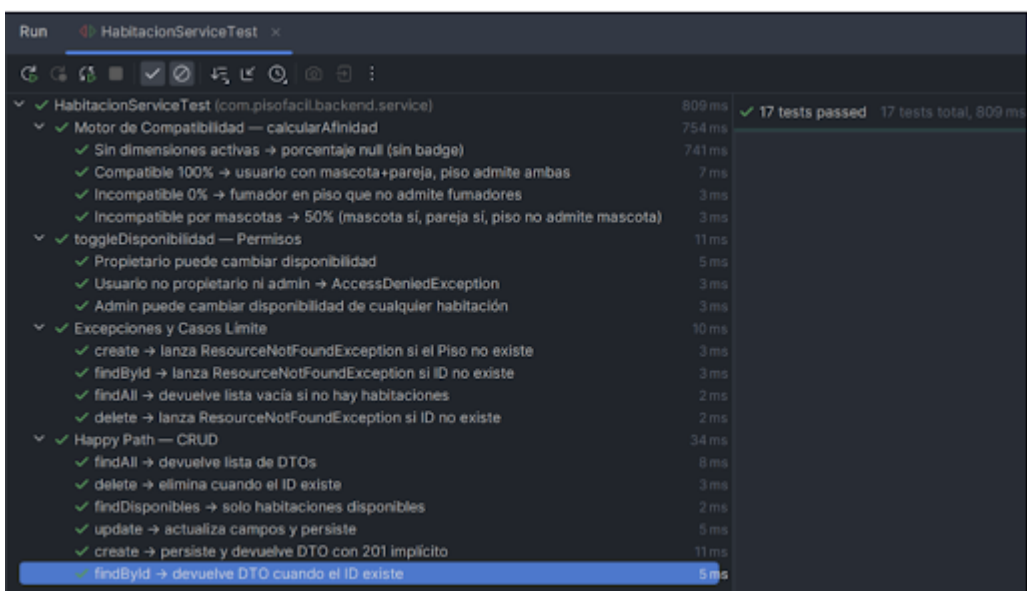
- **Tests de Infraestructura:** 1/1 (Carga del contexto de Spring)
- **Tests de Capa de Servicio:** 17/17 (Lógica y algoritmos)
- **Tests de Capa de Controlador:** 10/10 (API REST y validaciones)
- **Estado Final:** 100% PASS (28 tests totales ejecutados)

Aquí podemos ver las imágenes de los tests completados como evidencia

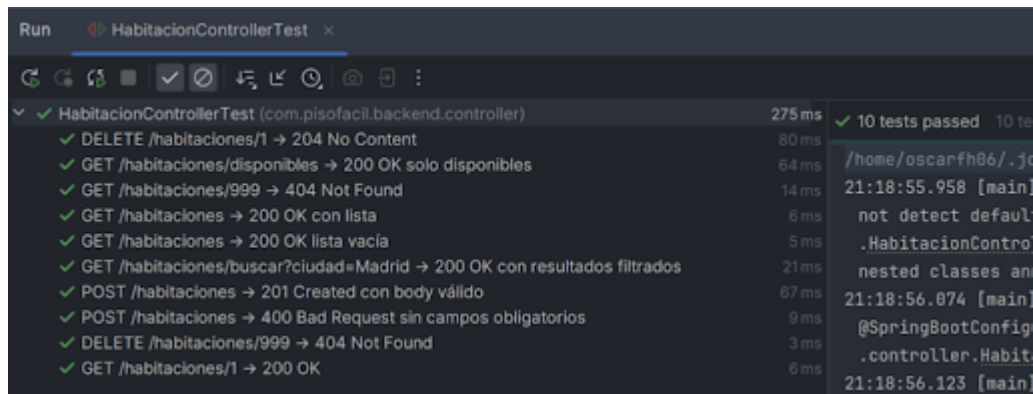
Test de Arquitectura



Test unitarios



Test de integración



La implementación de este plan de pruebas para la entidad Habitación garantiza que la plataforma PisoFácil cumple con los estándares de calidad exigidos:

- **Seguridad Robusta:** Se ha validado que solo los propietarios gestionan sus anuncios.
- **Precisión Algorítmica:** El motor de afinidad ha sido estresado con diferentes perfiles, asegurando que los usuarios encuentren compañeros compatibles.
- **Contrato API Estable:** Los desarrolladores de frontend tienen la garantía de que los endpoints devuelven siempre la estructura de datos acordada.

Una vez visto lo anterior, ya estarían finalizadas las pruebas